

- ElectroMacroDiff V5.2 Hybrid
 - Complete End-to-End Master Execution Blueprint
 - 1. Final Project Identity
 - 2. Real Constraints
 - 3. Final Deliverables
 - 4. Team Roles
 - 5. AI Tool Usage Strategy
 - 6. Google Drive Project Structure
 - 7. Required Notebooks
 - 8. Colab Requirements
 - 9. Dataset Plan
 - 9.1 Primary Datasets
 - 9.2 Protein Structures
 - 9.3 Training/Generation Sets
 - 10. Data Curation Rules
 - 11. File Schemas
 - 11.1 Curated Ligands
 - 11.2 Ligand Features
 - 11.3 SE(3) Graph Dataset
 - 11.4 Generated Candidates
 - 11.5 Docking Scores
 - 11.6 Final Ranking
 - 12. Module-by-Module Plan
 - M0 - Project Registry and Resume System
 - M1 - Data Collection and Curation
 - M2 - Ligand Features, Conformers, and SE(3) Dataset
 - M3 - Custom SE(3) Flow Matching Model
 - M4 - Hybrid Candidate Generation
 - M5 - Docking and Pose Filtering
 - M6 - ADMET, Safety Proxy, and Synthesizability
 - M7 - Consensus Ranking
 - M8 - TPP and Final Report
 - 13. 10-Day Execution Schedule
 - Day 1 - Setup and Environment
 - Day 2 - Data Collection
 - Day 3 - Features and SE(3) DataLoader
 - Day 4 - SE(3) Training and Baseline Generation

- Day 5 - SE(3) Inference and Candidate Merge
- Day 6 - Docking Setup and Controls
- Day 7 - Main Docking
- Day 8 - ADMET, Safety, Synthesis
- Day 9 - Ranking and Report Draft
- Day 10 - Verification and Presentation
- 14. 8-Day Compressed Schedule
- 15. Go/No-Go Gates
- 16. Risk Register
- 17. Evaluation Metrics
 - Generation Metrics
 - SE(3) Training Metrics
 - Docking Metrics
 - ADMET/Synthesis Metrics
- 18. Final Ranking Formula
- 19. What To Say In The Final Presentation
- 20. Immediate Day 1 Checklist
- 21. Source Notes

ElectroMacroDiff V5.2 Hybrid

Complete End-to-End Master Execution Blueprint

Version date: 2026-05-08

This is the final realistic plan for a 5-member B.Tech second-year team with 8-10 days, Google Colab free tier, 2-3 Google accounts per person, unreliable parallel execution, and Google Drive as the persistent storage layer.

The plan keeps the honest V5.2 execution strategy, but restores the research contribution: the custom SE(3) Flow Matching model is the primary candidate generator, while RDKit/SELFIES generation is the baseline and backup.

1. Final Project Identity

Project name: ElectroMacroDiff V5.2 Hybrid

Core claim:

We built a low-resource, checkpointed AI drug discovery pipeline centered on a custom SE(3) flow-matching molecular generator, supported by classical RDKit/SELFIES baselines, docking-based prioritization, ADMET filtering, synthesizability estimation, and final TPP-style reporting.

What the project should claim:

- custom deep learning generator was trained/tested under free Colab constraints
- pipeline is reproducible and resumable from Google Drive checkpoints
- generated molecules are computational candidates for experimental follow-up
- results are prioritization hypotheses, not clinical proof

What the project must not claim:

- experimentally proven JAK2 inhibitors
- clinically safe drugs
- FDA-approved or FDA-ready molecules
- guaranteed synthesis
- true wet-lab potency
- true FEP accuracy unless actual FEP was performed

2. Real Constraints

Team:

- 5 students
- second-year B.Tech level
- limited time for debugging heavy scientific software

Time:

- ideal: 10 days

- compressed: 8 days

Compute:

- Google Colab free tier
- T4 GPU when available
- CPU fallback required
- no assumption of stable long-running sessions
- no assumption of true parallel Colab execution

Storage:

- Google Drive is mandatory
- every output, checkpoint, log, and plot must be saved to Drive

Main technical risk:

- custom SE(3) model DataLoader/training may fail late unless tested early

Main strategic fix:

- SE(3) model is primary generator
- RDKit/SELFIES baseline continues in parallel as safety net

3. Final Deliverables

Minimum viable deliverables:

- curated JAK2 ligand dataset
- macrocycle/constrained-ligand analysis
- trained or at least debug-trained SE(3) model with checkpoints
- RDKit/SELFIES baseline generated molecules
- generated candidate CSV/SDF files
- docking scores for at least 50 molecules
- ADMET and synthesizability ranking for top candidates
- final ranked top 3-5 molecules
- TPP-style report
- presentation slides
- reproducible notebooks

- Google Drive artifact registry

Ideal deliverables:

- SE(3) trained 100-200 epochs or until stable convergence
- at least 100 valid SE(3)-generated candidates
- 500-2,000 baseline generated candidates
- 100-300 merged filtered candidates
- 50-150 docked candidates
- top 10 full molecule cards
- top 3-5 final candidates with 2D structure, 3D conformer, pose image, docking score, ADMET flags, synthesis score, and final recommendation

4. Team Roles

Member	Role	Main Responsibility	Backup Responsibility
Student 1	Integrator / Lead	Drive registry, notebooks, final merge, report structure	troubleshoot all modules
Student 2	Data Lead	ChEMBL/BindingDB/PDB collection, curation, splits	literature notes
Student 3	Model Lead	SE(3) DataLoader, training, checkpoints, inference	generation metrics
Student 4	Chemistry / Generation Lead	RDKit, SELFIES, descriptors, filters, SDF generation	SE(3) output cleanup
Student 5	Docking / Evaluation Lead	receptor prep, docking, ADMET, synthesis, final ranking	pose visualization

Daily rule:

- every member writes a short daily log
- every important file goes to Drive before sleeping
- no final result is allowed to exist only in Colab runtime

5. AI Tool Usage Strategy

Use AI tools as force multipliers, not as unsupervised truth sources.

Tool	Best Use
Codex / ChatGPT	code, debugging, notebook generation, pipeline organization
Gemini	literature summaries, Google/Colab workflow, target biology checks
Claude	report writing, methods clarity, limitation wording
DeepSeek / Qwen	alternative code implementations, tensor-shape debugging
Grok	brainstorming, quick sanity checks, presentation phrases
Antigravity / Codex	code organization, file generation, reproducible scripts

Rules:

- every AI-generated code cell must run on Tiny Debug Set first
- never paste unknown code directly into the main run
- save prompts and useful answers in `11_logs/ai_assistance_notes.md`
- AI tools can help write code; saved artifacts prove the project

6. Google Drive Project Structure

Create this exact folder:

```
EMD_V5_2_Hybrid/  
  00_project_registry/  
  01_raw_data/  
    chembl/  
    bindingdb/  
    pubchem/  
    pdb/  
    literature/  
  02_curated_data/  
  03_features/  
    ligand_graphs/
```

```
conformers/  
splits/  
04_models_checkpoints/  
se3_flow/  
baseline_models/  
05_generated_candidates/  
se3/  
selfies/  
rdkit/  
merged/  
06_docking/  
receptor/  
ligands_pdbqt/  
poses/  
scores/  
images/  
07_admet_synthesis/  
08_final_ranking/  
09_reports/  
10_notebooks/  
11_logs/
```

Every notebook begins with:

```
from google.colab import drive  
drive.mount("/content/drive")  
  
BASE = "/content/drive/MyDrive/EMD_V5_2_Hybrid"
```

7. Required Notebooks

```
10_notebooks/  
00_environment_and_drive_check.ipynb  
01_data_collection_and_curation.ipynb  
02_ligand_features_and_splits.ipynb  
03_se3_data_loader_training.ipynb  
04_candidate_generation_hybrid.ipynb  
05_docking_and_pose_filtering.ipynb  
06_admet_synthesis_safety.ipynb  
07_final_ranking_and_tpp.ipynb
```

Every notebook must contain:

- install cell

- Drive mount cell
 - config cell
 - Tiny Debug mode
 - resume/progress cell
 - main execution cell
 - output validation cell
 - summary cell
-

8. Colab Requirements

The Colab package list is saved separately:

- `requirements_v5_2_hybrid_colab.txt`

Core install cell:

```
!pip -q install -r  
"/content/drive/MyDrive/EMD_V5_2_Hybrid/requirements_v5_2_hybrid_colab.txt"
```

If installing from local workspace first, upload/copy the requirements file into:

```
EMD_V5_2_Hybrid/00_project_registry/requirements_v5_2_hybrid_colab.txt
```

Main packages:

- RDKit
- SELFIES
- ChEMBL webresource client
- Biopython
- PyTorch
- e3nn
- torchdiffeq
- Vina/Meeko where possible
- matplotlib/seaborn/py3Dmol
- python-docx/WeasyPrint for reporting

Optional stretch packages:

- GNINA binary
- DiffDock from GitHub
- AiZynthFinder
- torch-geometric

Fallback principle:

- if a package takes more than 2 hours to install/debug, move to fallback and protect the pipeline

9. Dataset Plan

9.1 Primary Datasets

Dataset	Role	Required Output
ChEMBL 36	primary JAK2 bioactivity data	curated ligand/activity table
BindingDB	additional binding/activity records	merged activity backup
RCSB PDB	JAK2 protein structures	receptor PDB/mmCIF files
PubChem	backup molecule metadata and activity notes	optional metadata table

As of the latest checked official ChEMBL download documentation, ChEMBL 36 is the current release listed for July 2025. Use ChEMBL 36 where possible; use the ChEMBL API if downloading the full database is too slow.

9.2 Protein Structures

Primary receptor:

- JAK2 crystal structure, recommended starting structure: PDB 5AEP

Backup/optional:

- additional JAK2 kinase domain structures from RCSB
- JAK1/JAK3/TYK2/FLT3 structures for selectivity if time allows

Pocket:

- ATP-binding pocket or co-crystallized ligand pocket
- grid centered around reference ligand or active-site residues

9.3 Training/Generation Sets

Set	Target Size	Purpose
Tiny Debug Set	20-30 molecules	test every notebook quickly
SE(3) Sprint Train Set	100-700 molecules	train custom model
Main Curated Set	300-1,500 molecules	descriptors, baselines, ranking
Baseline Generation Seeds	50-200 potent/scaffold-diverse molecules	RDKit/SELFIES generation
Docking Set	50-150 molecules	realistic docking workload
Final Evaluation Set	top 10-20 molecules	ADMET/synthesis/full cards

If true macrocyclic JAK2 ligands are too few:

- include constrained JAK2 ligands
- include potent JAK2 ligands with macrocycle-like linker potential
- clearly label this as "macrocycle/constrained inhibitor generation"

10. Data Curation Rules

Keep activity records when:

- target is JAK2 or clearly maps to human JAK2
- activity type is IC50, Ki, Kd, or inhibition with usable context
- units can be converted to nM or a comparable value
- SMILES is present and RDKit-sanitizable
- assay confidence is acceptable

Preferred fields:

```
source
source_compound_id
chembl_id
bindingdb_id
pubchem_cid
canonical_smiles
standard_type
standard_relation
standard_value
standard_units
standard_value_nM
p_activity
target_name
target_id
assay_id
assay_description
confidence_score
doi_or_reference
```

Deduplication:

- canonicalize SMILES
- compute InChIKey
- group by InChIKey
- retain median potency or best-confidence potency
- keep source provenance

Macrocycle/constrained labels:

```
max_ring_size
has_macrocycle_12_20
has_large_ring_8_11
num_rings
num_rotatable_bonds
murcko_scaffold
```

Recommended filters:

- molecular weight: 250-900 for broad macrocycle/constrained search
 - ring size: 8-24 for analysis, 12-20 for strict macrocycle flag
 - remove salts/mixtures
 - remove invalid valence structures
 - flag but do not immediately remove PAINS until final filtering
-

11. File Schemas

11.1 Curated Ligands

File:

```
02_curated_data/jak2_curated_ligands.csv
```

Columns:

```
mol_id
source
source_id
canonical_smiles
inchikey
activity_type
activity_value_nM
p_activity
target
assay_id
confidence_score
max_ring_size
has_macrocycle_12_20
has_constrained_ring_8_11
split
notes
```

11.2 Ligand Features

File:

```
03_features/ligand_features.csv
```

Columns:

```
mol_id  
canonical_smiles  
mw  
logp  
tpsa  
hbd  
hba  
rotatable_bonds  
qed  
num_atoms  
num_heavy_atoms  
num_rings  
max_ring_size  
formal_charge  
passes_rdkit  
passes_basic_filters
```

11.3 SE(3) Graph Dataset

Files:

```
03_features/ligand_graphs/se3_graphs.pt  
03_features/ligand_graphs/se3_graph_index.csv
```

Each graph should contain:

```
atom_features: [num_atoms, atom_feature_dim]  
bond_index: [2, num_edges]  
bond_features: [num_edges, bond_feature_dim]  
coords: [num_atoms, 3]  
mol_id  
smiles  
ring_mask  
macrocycle_mask  
activity_label optional
```

Atom features:

- atomic number embedding/index
- formal charge
- aromatic flag
- hybridization
- degree
- valence
- ring membership
- macrocycle ring flag
- donor flag
- acceptor flag
- Gasteiger charge
- chirality flag

Bond features:

- single/double/triple/aromatic
- conjugated flag
- ring bond flag
- stereo flag

11.4 Generated Candidates

File:

```
05_generated_candidates/merged/generated_merged_filtered.csv
```

Columns:

```
candidate_id
source_generator
parent_mol_id
canonical_smiles
inchikey
valid_rdkit
unique_flag
novel_flag
mw
logp
tpsa
qed
sa_score
```

```
max_ring_size
has_macrocycle_12_20
generation_notes
```

11.5 Docking Scores

File:

```
06_docking/scores/docking_scores.csv
```

Columns:

```
candidate_id
smiles
docking_engine
receptor_pdb
grid_center_x
grid_center_y
grid_center_z
grid_size_x
grid_size_y
grid_size_z
best_score
pose_rank
pose_file
pose_image
control_or_generated
notes
```

11.6 Final Ranking

File:

```
08_final_ranking/final_ranked_candidates.csv
```

Columns:

```
rank
candidate_id
source_generator
```

```
smiles
inchikey
docking_score
pose_score
qed
sa_score
admet_score
safety_proxy_score
novelty_score
diversity_cluster
final_weighted_score
decision
main_risk
```

12. Module-by-Module Plan

M0 - Project Registry and Resume System

Purpose:

- prevent Colab disconnects from destroying work

Input:

- project base path
- team member list
- campaign config

Output:

```
00_project_registry/campaign_config.yaml
00_project_registry/run_registry.csv
00_project_registry/progress_*.json
00_project_registry/environment_versions.txt
```

Required config:

```
project_name: EMD_V5_2_Hybrid
target: JAK2
primary_pdb: 5AEP
random_seed: 42
tiny_debug: true
```



```
max_train_molecules: 700
max_generated_se3: 500
max_generated_baseline: 2000
max_docking_molecules: 150
drive_base: /content/drive/MyDrive/EMD_V5_2_Hybrid
```

Resume rule:

- every long loop writes `progress_*.json`
- every model run writes latest and best checkpoint
- every generated batch is appended to disk

Generic progress pattern:

```
import os, json, time

def load_progress(path):
    if os.path.exists(path):
        return json.load(open(path))
    return {"last_completed_index": -1}

def save_progress(path, progress):
    progress["updated_at"] = time.strftime("%Y-%m-%d %H:%M:%S")
    json.dump(progress, open(path, "w"), indent=2)
```

Success gate:

- disconnect and reconnect test works on Tiny Debug Set

M1 - Data Collection and Curation

Purpose:

- build the training and screening dataset

Inputs:

- ChEMBL 36 API/download
- BindingDB export
- RCSB PDB JAK2 structures
- optional PubChem metadata

Core JAK2 search terms:

JAK2
Janus kinase 2
Tyrosine-protein kinase JAK2
ChEMBL target records mapped to human JAK2

Process:

1. Query target records.
2. Pull activities for IC50/Ki/Kd.
3. Convert units to nM.
4. Convert to pActivity.
5. Canonicalize SMILES.
6. Remove duplicates.
7. Compute ring labels.
8. Split into train/val/test by scaffold where possible.

Outputs:

```
01_raw_data/chembl/jak2_activities_raw.csv
01_raw_data/bindingdb/jak2_bindingdb_raw.tsv
01_raw_data/pdb/5AEP.pdb
02_curated_data/jak2_curated_ligands.csv
02_curated_data/jak2_macrocycle_constrained.csv
02_curated_data/jak2_train_val_test_split.csv
```

Success gate:

- at least 300 usable JAK2/constrained ligands if possible
- at least 20-30 Tiny Debug molecules
- receptor PDB file downloaded

Fallback:

- if macrocycles are too sparse, include constrained and potent JAK2 inhibitors and state the scope honestly

M2 - Ligand Features, Conformers, and SE(3) Dataset

Purpose:

- prepare both classical descriptors and model tensors

Inputs:

```
02_curated_data/jak2_curated_ligands.csv
```

Process:

1. RDKit sanitization.
2. Salt stripping/standardization.
3. Descriptor calculation.
4. Macrocycle/ring analysis.
5. Gasteiger charge calculation.
6. 3D conformer generation.
7. MMFF/UFF minimization.
8. Graph tensor creation.

Descriptors:

- MW
- logP
- TPSA
- HBD
- HBA
- rotatable bonds
- QED
- ring count
- max ring size
- formal charge

Outputs:

```
03_features/ligand_features.csv
03_features/conformers/curated_conformers.sdf
03_features/ligand_graphs/se3_graphs.pt
03_features/ligand_graphs/se3_graph_index.csv
03_features/splits/train_ids.txt
03_features/splits/val_ids.txt
03_features/splits/test_ids.txt
```

Success gate:

- more than 90% of curated molecules pass RDKit sanitization
- at least 100 graph tensors available for model debug/training

Fallback:

- if conformer generation fails for many macrocycles, reduce conformers per molecule and use UFF fallback

M3 - Custom SE(3) Flow Matching Model

Purpose:

- preserve the deep learning innovation

Inputs:

```
03_features/ligand_graphs/se3_graphs.pt
03_features/splits/train_ids.txt
03_features/splits/val_ids.txt
```

Model role:

- learn a 3D molecular generation or coordinate-flow process over ligand graphs/conformers
- use electronic/atom/bond features as conditioning
- generate candidate structures or reconstruct/generate coordinate proposals

Recommended architecture contract:

Input:

```
atom_features [B, N, F_atom]
coords_t [B, N, 3]
bond_index / adjacency
bond_features
time_t [B]
mask [B, N]
```

Output:

```
velocity/noise/displacement [B, N, 3]
optional atom logits or validity heads
```

Loss:

```
flow matching MSE or denoising MSE
optional bond/geometry regularization
```

Training settings for Colab T4:

```
epochs: 100-200 target, stop earlier if unstable
batch_size: 2-8 depending on memory
learning_rate: 1e-4 to 3e-4
weight_decay: 1e-5
gradient_clip: 1.0
precision: fp32 first, mixed precision only after stable
checkpoint_every: 10-20 epochs
validation_every: 5-10 epochs
```

Required debug tests before training:

- import model
- load one graph
- collate one batch
- forward pass
- finite loss
- backward pass
- optimizer step
- save checkpoint
- reload checkpoint
- inference on one sample

Checkpoint format:

```
checkpoint = {
    "epoch": epoch,
```

```
"global_step": global_step,
"model_state_dict": model.state_dict(),
"optimizer_state_dict": optimizer.state_dict(),
"scheduler_state_dict": scheduler.state_dict() if scheduler else None,
"train_loss": float(train_loss),
"val_loss": float(val_loss),
"config": config,
"random_seed": seed,
}
```

Outputs:

```
04_models_checkpoints/se3_flow/se3_latest_checkpoint.pt
04_models_checkpoints/se3_flow/se3_best_checkpoint.pt
04_models_checkpoints/se3_flow/se3_epoch_XXXX.pt
04_models_checkpoints/se3_flow/training_log.csv
04_models_checkpoints/se3_flow/training_curve.png
```

Success gate:

- at minimum: one successful train step and checkpoint reload
- good: 100 epochs completed
- ideal: stable validation curve and valid generated molecules

Hard go/no-go rule:

- if M3 cannot complete one optimizer step by end of Day 3, freeze SE(3) as architecture demo and use RDKit/SELFIES as main molecule source

M4 - Hybrid Candidate Generation

Purpose:

- generate candidates from both custom model and baseline methods

Inputs:

```
04_models_checkpoints/se3_flow/se3_best_checkpoint.pt
03_features/ligand_features.csv
02_curated_data/jak2_curated_ligands.csv
```

SE(3) generation:

1. Load best checkpoint.
2. Sample or perturb molecule/conformer seeds.
3. Decode/convert outputs to valid molecular representations where applicable.
4. Sanitize with RDKit.
5. Save all raw and filtered results.

Baseline generation:

- SELFIES mutation around potent known ligands
- RDKit substituent replacement
- fragment recombination
- ring/linker modification where chemically valid

Filtering:

- RDKit validity
- uniqueness
- novelty versus training set
- ring size
- MW/logP/TPSA bounds
- QED
- remove obvious PAINS only at later stage unless severe

Outputs:

```
05_generated_candidates/se3/generated_se3_raw.csv
05_generated_candidates/se3/generated_se3_filtered.csv
05_generated_candidates/se3/generated_se3_filtered.sdf
05_generated_candidates/selfies/generated_selfies_filtered.csv
05_generated_candidates/rdkit/generated_rdkit_filtered.csv
05_generated_candidates/merged/generated_merged_filtered.csv
05_generated_candidates/merged/generation_comparison_metrics.csv
```

Generation metrics:

- number attempted
- number valid
- validity percentage
- uniqueness
- novelty

- max ring size distribution
- QED distribution
- SA score distribution
- generator source representation in top candidates

Success gate:

- at least 100 filtered candidates total
 - at least some SE(3)-origin candidates included if possible
-

M5 - Docking and Pose Filtering

Purpose:

- prioritize candidates by approximate binding pose/score against JAK2

Inputs:

```
05_generated_candidates/merged/generated_merged_filtered.csv
01_raw_data/pdb/5AEP.pdb
```

Recommended docking hierarchy:

1. AutoDock Vina: most practical fallback
2. GNINA: better if binary works
3. DiffDock: stretch if setup succeeds quickly

Controls:

- dock at least 3-5 known JAK2 ligands first
- include known ligands in score table as controls

Receptor preparation:

- remove waters unless required
- keep relevant cofactor/ions only if justified
- add hydrogens
- assign charges where tool requires
- define grid around co-crystallized ligand/pocket

Candidate preparation:

- convert SMILES/SDF to 3D
- minimize
- convert to docking format if needed

Outputs:

```
06_docking/receptor/jak2_prepared.pdbqt
06_docking/ligands_pdbqt/
06_docking/poses/
06_docking/scores/docking_scores.csv
06_docking/images/top_pose_*.png
```

Success gate:

- 50 candidates docked minimum
- top 10 poses visually inspected

Fallback:

- if docking fails, reduce to top 30 candidates and use simpler Vina settings
-

M6 - ADMET, Safety Proxy, and Synthesizability

Purpose:

- remove candidates that look impossible, toxic, or un-drug-like

Inputs:

```
06_docking/scores/docking_scores.csv
05_generated_candidates/merged/generated_merged_filtered.csv
```

Computed filters:

- QED
- SA score

- Lipinski rules
- Veber rules
- PAINS
- Brenk alerts if available
- MW/logP/TPSA/HBD/HBA/rotatable bonds

SLC19A3 proxy:

- do not claim true SLC19A3 safety
- use literature note + similarity/off-target proxy if available
- flag candidates that resemble known problematic chemotypes if identified

Synthesizability:

- SA score for all top candidates
- AiZynthFinder for top 5-10 only if installation succeeds quickly
- otherwise provide preliminary route plausibility notes

Outputs:

```
07_admet_synthesis/admet_scores.csv
07_admet_synthesis/filter_flags.csv
07_admet_synthesis/safety_proxy_notes.csv
07_admet_synthesis/synthesis_scores.csv
07_admet_synthesis/aizynth_routes/ optional
```

Success gate:

- top 10 candidates fully scored

M7 - Consensus Ranking

Purpose:

- combine generation, docking, chemistry, safety, and synthesis into final ranked candidates

Inputs:

```
05_generated_candidates/merged/generation_comparison_metrics.csv
06_docking/scores/docking_scores.csv
07_admet_synthesis/admet_scores.csv
07_admet_synthesis/synthesis_scores.csv
```

Default weights:

```
docking score: 35%
pose sanity: 15%
ADMET/drug-likeness: 20%
synthesizability: 15%
novelty/diversity: 10%
safety proxy: 5%
```

Rules:

- do not select all final molecules from the same scaffold
- include at least one SE(3)-generated molecule in final discussion if scientifically defensible
- if SE(3) molecules score poorly, say so honestly and present it as early-stage model learning

Outputs:

```
08_final_ranking/final_ranked_candidates.csv
08_final_ranking/top10_molecule_cards.xlsx
08_final_ranking/final_top5.sdf
```

Success gate:

- 3-5 final proposed molecules
- 5-10 backup molecules

M8 - TPP and Final Report

Purpose:

- package work into a defensible academic/research submission

Inputs:

```
08_final_ranking/final_ranked_candidates.csv
06_docking/images/
07_admet_synthesis/
04_models_checkpoints/se3_flow/training_curve.png
```

Final report sections:

1. Abstract
2. Problem statement
3. Why JAK2 and macrocyclic/constrained inhibitors
4. Dataset sources and curation
5. Custom SE(3) Flow Matching model
6. Baseline RDKit/SELFIES generation
7. Docking protocol
8. ADMET/safety/synthesis protocol
9. Results
10. SE(3) vs baseline comparison
11. Final top molecules
12. Limitations
13. Future work

TPP molecule card fields:

- candidate ID
- SMILES
- 2D structure
- 3D pose image
- generator source
- docking score
- QED
- SA score
- ADMET flags
- safety proxy notes
- synthesis notes
- final decision

Outputs:

```
09_reports/EMD_V5_2_Hybrid_Final_Report.pdf
09_reports/EMD_V5_2_Hybrid_TPP.pdf
09_reports/EMD_V5_2_Hybrid_Presentation.pptx
09_reports/EMD_V5_2_Hybrid_Audit_Package.zip
```

13. 10-Day Execution Schedule

Day 1 - Setup and Environment

Goals:

- create Drive folder
- upload requirements
- create notebooks
- run environment check
- run Tiny Debug path with 5-20 molecules

Outputs:

- `environment_versions.txt`
- `campaign_config.yaml`
- Tiny Debug folder populated

Owner:

- Student 1, all assist

Day 2 - Data Collection

Goals:

- collect JAK2 ligands from ChEMBL/BindingDB
- download PDB 5AEP
- create raw and curated CSV

Outputs:

- `jak2_curated_ligands.csv`
- `5AEP.pdb`

Owner:

- Student 2

Day 3 - Features and SE(3) DataLoader

Goals:

- compute descriptors
- build graph tensors
- fix DataLoader
- complete one SE(3) train step
- save/reload checkpoint

Outputs:

- `ligand_features.csv`
- `se3_graphs.pt`
- `se3_data_loader_test_passed.json`
- `se3_latest_checkpoint.pt`

Owner:

- Student 3 + Student 4

Day 4 - SE(3) Training and Baseline Generation

Goals:

- train SE(3) for 100-200 epochs if stable
- save checkpoints every 10-20 epochs
- generate SELFIES/RDKit baseline candidates

Outputs:

- `se3_best_checkpoint.pt`

- `training_curve.png`
- `generated_selfies_filtered.csv`
- `generated_rdkit_filtered.csv`

Owner:

- Student 3 and Student 4

Day 5 - SE(3) Inference and Candidate Merge

Goals:

- generate SE(3) candidates
- sanitize/filter candidates
- merge SE(3), SELFIES, RDKit
- compute generation metrics

Outputs:

- `generated_se3_filtered.csv`
- `generated_merged_filtered.csv`
- `generation_comparison_metrics.csv`

Owner:

- Student 3 + Student 4

Day 6 - Docking Setup and Controls

Goals:

- prepare JAK2 receptor
- prepare known ligands and generated ligands
- dock known control ligands
- dock first candidate batch

Outputs:

- `jak2_prepared.pdbqt`
- control docking scores

- first candidate poses

Owner:

- Student 5

Day 7 - Main Docking

Goals:

- dock 50-150 candidates
- save poses
- generate top pose images

Outputs:

- `docking_scores.csv`
- `top_pose_*.png`

Owner:

- Student 5

Day 8 - ADMET, Safety, Synthesis

Goals:

- compute ADMET descriptors
- run PAINS/Brenk checks
- compute SA/QED
- attempt AiZynthFinder for top 5-10 if feasible
- write SLC19A3 proxy notes

Outputs:

- `admet_scores.csv`
- `synthesis_scores.csv`
- `safety_proxy_notes.csv`

Owner:

- Student 5 + Student 1

Day 9 - Ranking and Report Draft

Goals:

- final weighted ranking
- top 10 molecule cards
- SE(3) vs baseline comparison
- draft report and TPP

Outputs:

- `final_ranked_candidates.csv`
- draft report
- draft TPP

Owner:

- Student 1, all assist

Day 10 - Verification and Presentation

Goals:

- rerun Tiny Debug pipeline
- verify all Drive links/files
- polish report/slides
- rehearse explanation

Outputs:

- final PDF report
- TPP PDF
- presentation
- audit package

Owner:

- all
-

14. 8-Day Compressed Schedule

If only 8 days are available:

Day	Work
1	setup + data collection start
2	curation + PDB + features
3	SE(3) DataLoader + one train step + baseline generation
4	SE(3) training/inference + merge candidates
5	receptor prep + control docking + first candidate docking
6	main docking + ADMET/synthesis filters
7	final ranking + report/TPP draft
8	verification + slides + final package

Compressed-mode sacrifice:

- skip DiffDock/GNINA if setup is slow
- use Vina
- use SA score instead of full AiZynthFinder if needed
- train SE(3) for fewer epochs but preserve the model story with checkpoints and curves

15. Go/No-Go Gates

Deadline	Gate	If Pass	If Fail
End Day 1	Drive + environment works	continue	fix before anything else
End Day 2	curated data exists	continue	reduce data ambition
End Day 3	SE(3) one train step works	train model	freeze as demo, use baseline candidates

Deadline	Gate	If Pass	If Fail
End Day 5	merged candidates exist	dock	use known ligands + baseline candidates
End Day 6	docking controls work	main docking	simplify receptor/docking
End Day 8	top 10 scored	report	rank with available evidence
End Day 10	final package complete	submit	submit minimum viable package

16. Risk Register

Risk	Probability	Impact	Mitigation
Colab disconnects	high	high	Drive checkpoint every batch/epoch
SE(3) DataLoader shape bugs	high	high	Day 3 dedicated debug day
Too few macrocyclic JAK2 ligands	medium	high	include constrained ligands and be honest
Docking install failure	medium	high	Vina fallback
GNINA/DiffDock too heavy	high	medium	stretch only
AiZynthFinder too heavy	high	medium	SA score + route notes fallback
Generated molecules invalid	medium	high	SELFIES/RDKit backup
Reviewer asks about novelty	high	high	show SE(3) architecture, training curve, comparison
Reviewer asks about proof	high	medium	state computational hypothesis limits clearly

17. Evaluation Metrics

Generation Metrics

- validity percentage
- uniqueness percentage
- novelty percentage
- macrocycle/constrained ring retention
- mean/max ring size
- descriptor distributions
- number selected for docking
- number in final top 10

SE(3) Training Metrics

- training loss
- validation loss
- gradient stability
- checkpoint count
- inference success count
- generated validity

Docking Metrics

- docking success rate
- best score
- pose rank
- control ligand score comparison
- pose visual sanity

ADMET/Synthesis Metrics

- QED
- SA score
- Lipinski violations

- Veber violations
 - PAINS/Brenk flags
 - safety proxy notes
 - route found or not found if AiZynthFinder works
-

18. Final Ranking Formula

Default:

```
final_score =  
    0.35 * docking_score_norm  
+ 0.15 * pose_sanity_norm  
+ 0.20 * admet_norm  
+ 0.15 * synthesis_norm  
+ 0.10 * novelty_diversity_norm  
+ 0.05 * safety_proxy_norm
```

Selection rules:

- choose top 3-5 final candidates
 - keep 5 backups
 - do not choose duplicates or near-identical scaffolds
 - do not hide baseline results
 - compare SE(3) and baseline honestly
-

19. What To Say In The Final Presentation

Strong wording:

ElectroMacroDiff V5.2 Hybrid demonstrates that a student team can build a low-resource, reproducible AI drug discovery workflow by combining a custom SE(3) flow-matching molecular generator with classical cheminformatics baselines, docking, ADMET filtering, and TPP-style reporting.

When asked about deep learning:

The deep learning core is our SE(3) flow-matching generator. It uses 3D molecular/electronic graph features and was trained under free Colab constraints. RDKit and SELFIES were included as baselines and fallbacks, not as replacements.

When asked about limitations:

The outputs are computationally prioritized candidates. They require synthesis, biochemical assays, and safety testing before any biological claim can be made.

20. Immediate Day 1 Checklist

Do this first:

1. Create Google Drive folder `EMD_V5_2_Hybrid`.
2. Create all subfolders.
3. Upload `requirements_v5_2_hybrid_colab.txt`.
4. Create `00_environment_and_drive_check.ipynb`.
5. Test RDKit import.
6. Test PyTorch GPU availability.
7. Test loading one SMILES and computing descriptors.
8. Test saving a CSV to Drive.
9. Test reading it back.
10. Write first daily log.

Only after that, start data collection.

21. Source Notes

Use official or primary sources where possible:

- ChEMBL downloads and release documentation
- ChEMBL official Python webresource client
- BindingDB official data/download pages
- RCSB PDB and RCSB Data API
- RDKit official documentation

- SELFIES official documentation/paper
- AutoDock Vina documentation
- GNINA GitHub/papers if used
- DiffDock GitHub/paper if used
- AiZynthFinder paper/GitHub if used

All final reports should cite the exact tools and data versions actually used.